

Melhorias Realizadas

(NotebookLM)

Avaliação proposta para a LLM:

Realize uma descrição objetiva das melhorias realizadas do arquivo 1 (Problema: Análise de Desempenho e Custo de Manutenção em Frotas de Bombas Centrífugas) para o arquivo 2 (Evolução Técnica da Solução Desenvolvida na Atividade Anterior). Além disso, avalie as limitações do arquivo 1 e próximos passos para melhor eficiência.

Resposta:

A evolução entre o arquivo original (Arquivo 1) e a nova especificação técnica (Arquivo 2) demonstra um amadurecimento significativo da ferramenta, saindo de um conceito de cálculo básico para uma solução estruturada de suporte à decisão.

1. Melhorias Realizadas (Arquivo 1 para Arquivo 2)

As principais melhorias identificadas na evolução técnica incluem:

- **Rigor Técnico e Validação de Dados:** O Arquivo 2 introduz regras de validação estritas para os inputs (ex: rendimento entre 1% e 100%, horas de operação entre 0,1 e 24,0, e TAGs únicos), garantindo a integridade dos cálculos e evitando erros comuns de preenchimento.
- **Refinamento do Processamento:** Foi adicionada uma nota técnica explicando que o rendimento reduzido aumenta a demanda de potência da rede para o mesmo trabalho, ajustando a percepção do consumo real.
- **Inteligência de Dados (Insights Automáticos):** Além de exibir dados individuais, a nova versão propõe o cálculo do "**Custo da Ineficiência**", que soma o prejuízo diário de todos os ativos na zona crítica, e a identificação do "**Alvo Crítico**" para priorização de manutenção.
- **Experiência do Usuário (UX):** A evolução prevê funcionalidades práticas como **botões de exportação** (.csv ou .xlsx) e **filtros dinâmicos** para frotas grandes, facilitando o trabalho de análise.
- **Exemplificação Prática:** O Arquivo 2 inclui um cenário de teste com três bombas (BC-201, BC-202 e BC-203), permitindo validar se a lógica de criticidade (verde, amarelo e vermelho) está funcionando conforme o esperado.

2. Avaliação das Limitações do Arquivo 2

Apesar do avanço, o Arquivo 2 ainda apresenta limitações que podem impactar sua eficiência em escala real:

1. **Dependência de Input Manual:** O sistema ainda exige a inserção manual de dados técnicos e operacionais, o que aumenta a carga de trabalho e o risco de erro humano em plantas com centenas de bombas.
2. **Análise Estática (Snapshot):** A ferramenta analisa o estado atual ("fotografia"), mas não possui memória histórica para realizar análise de tendências de degradação ao longo do tempo.
3. **Custo de Manutenção Simplificado:** O custo de manutenção é tratado como um valor fixo por hora operacional, ignorando variáveis reais como custos de peças de reposição, logística ou paradas não planejadas.

4. **Falta do Cálculo de Payback:** Embora o documento mencione a priorização baseada em retorno financeiro (Payback), as fórmulas para calcular o custo da reforma versus a economia gerada não foram explicitamente detalhadas no processamento.

Próximos Passos para Melhor Eficiência

Para elevar a solução a um nível de excelência em manutenção preditiva e gestão de energia, os seguintes passos são recomendados:

- **Integração com Sensores (IoT/SCADA):** Automatizar a coleta de vazão, potência e horas de operação diretamente dos sistemas de controle da planta, eliminando o preenchimento manual.
- **Módulo de Predição de Falhas:** Implementar algoritmos que utilizem o histórico de rendimento para prever em quanto tempo uma bomba atingirá o nível crítico (70%), permitindo o planejamento antecipado da parada.
- **Cálculo Automatizado de ROI:** Adicionar um campo para "Custo Estimado de Reparo" e criar uma função que calcule automaticamente em quantos meses a economia de energia pagará o investimento na manutenção do ativo.
- **Alertas e Notificações:** Configurar o sistema para enviar alertas automáticos por e-mail ou via sistemas de gestão (como SAP ou Totvs) assim que um ativo entrar na zona de "Alto Risco".

Avaliação proposta para a LLM:

Realize uma descrição objetiva das melhorias realizadas do Código 1 (Resolução Gemini) para o Código 2 (Código Gemini). Além disso, avalie

- Qualidade técnica da solução;
- Clareza, utilidade e usabilidade para o usuário final;
- Melhoria da interface ou da forma de interação;
- Qualidade das visualizações;
- Robustez, confiabilidade e organização do código;

Resposta:

A transição do **Código 1 (Resolução Gemini)** para o **Código 2 (Código Gemini)** apresenta uma evolução significativa em termos de maturidade de software e aplicação de engenharia.

Abaixo, apresento a descrição objetiva das melhorias e a avaliação detalhada conforme solicitado:

Descrição Objetiva das Melhorias

- **Validação de Dados:** O Código 2 introduz uma função dedicada (`obter_input_valido`) que utiliza laços `while` e blocos `try/except` para garantir que as entradas sejam numéricas e estejam dentro de limites técnicos (mínimo e máximo), evitando que o programa encerre por erro do usuário.
- **Integridade dos Ativos:** Implementação da função `obter_tag_unica`, que utiliza um conjunto (`set`) para impedir o cadastro de bombas com TAGs duplicadas ou vazias, algo essencial para o gerenciamento de inventário industrial.

- **Modularização:** O Código 2 separa as responsabilidades em funções menores e especializadas (entrada, classificação, dashboard, exportação), facilitando a manutenção e leitura.
- **Funcionalidades de Pós-Processamento:** Foram adicionadas opções de **filtro dinâmico** para visualizar apenas ativos críticos e a **exportação de dados** para formatos profissionais como Excel (.xlsx) e CSV.
- **Inteligência de Negócio:** Criação de "Insights Automáticos", como o cálculo do **"Custo da Ineficiência"**, que soma o impacto financeiro apenas dos ativos na zona de alto risco.

Avaliação da Solução

1. Qualidade Técnica da Solução

O Código 2 demonstra uma qualidade técnica superior ao utilizar bibliotecas avançadas como `numpy` para manipulação de eixos e `seaborn` para estética estatística. A lógica de engenharia é mais refinada, pois utiliza as propriedades de DataFrames do `pandas` para ordenação de rankings e somatórios segmentados, indo além da simples coleta e exibição de dados.

2. Clareza, Utilidade e Usabilidade para o Usuário Final

A usabilidade foi drasticamente melhorada. Enquanto o Código 1 é linear e rígido, o Código 2 fornece um fluxo interativo. A utilidade para um engenheiro é maior no Código 2, pois ele traduz dados técnicos em decisões financeiras (potencial de economia). As mensagens de erro são claras e orientam o usuário sobre o que foi digitado incorretamente.

3. Melhoria da Interface e Forma de Interação

A interface no terminal tornou-se mais limpa com a função `limpar_tela()`, que organiza as etapas de cadastro e resultados. A interação não termina na exibição dos gráficos; o sistema permite que o usuário decida se quer filtrar a tabela ou salvar os arquivos, simulando o comportamento de um software profissional.

4. Qualidade das Visualizações

As visualizações no Código 2 são mais profissionais e padronizadas:

- **Cores de Risco:** Utiliza uma paleta fixa (Verde/Amarelo/Vermelho) vinculada aos níveis de criticidade, facilitando a leitura rápida (gestão visual).
- **Layout:** O uso de um grid 2x2 organiza melhor as informações, e o gráfico de custos foi transformado em um **Ranking**, o que é mais útil para priorização de manutenção do que um gráfico de barras simples.
- **Robustez Visual:** O gráfico de pizza no Código 2 é mapeado apenas para categorias existentes, evitando erros de exibição caso nem todos os níveis de risco estejam presentes na amostra.

5. Robustez, Confiabilidade e Organização do Código

O Código 2 é consideravelmente mais robusto. Ele trata falhas comuns de entrada que derrubariam o Código 1 (como digitar letras onde se esperam números ou valores fora de escala). A organização segue melhores práticas de Python, utilizando o bloco `if __name__ == "__main__":` e um ponto de entrada centralizado (`main()`), o que torna o código confiável para ser integrado a sistemas maiores.